

# ui.download 全面详细解析

`ui.download` 是 NiceGUI 框架 2.14.0 版本新增的文件下载工具集，专注于简化文件下载操作，支持本地文件、网络资源（URL）、原始数据（字符串 / 字节）三类下载场景，提供简洁统一的 Python API，无需手动处理 HTTP 响应或浏览器下载逻辑，与 NiceGUI 组件生态深度兼容。

## 一、核心功能概览

`ui.download` 围绕“文件下载”核心需求，提供三大核心方法，覆盖主流下载场景：

- `ui.download.file()`：从本地文件路径下载文件；
- `ui.download.from_url()`：从指定 URL 下载资源（支持相对路径，有限支持绝对路径）；
- `ui.download.content()`：直接下载原始字符串或字节数据（动态生成文件）。

所有方法均支持通过按钮点击等交互事件触发，无需复杂配置，开箱即用。

## 二、详细功能说明与使用示例

### 1. 公共参数说明

三大核心方法共享三个可选参数，用于控制下载文件的名称和类型，参数规则完全一致：

| 参数名                     | 类型  | 说明                                                                                                 |
|-------------------------|-----|----------------------------------------------------------------------------------------------------|
| <code>filename</code>   | 字符串 | 下载文件的自定义名称（含扩展名），默认使用源文件 / 资源的原始名称（如服务器返回的文件名）                                                     |
| <code>media_type</code> | 字符串 | 文件的 MIME 类型（如 <code>text/plain</code> 、 <code>application/json</code> ），默认自动推断，可手动指定以确保浏览器正确识别文件类型 |

示例：下载文本文件时指定 `media_type='text/plain'`，下载 Excel 文件时指定 `media_type='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'`。

### 2. 从本地文件路径下载 (`ui.download.file()`)

#### 功能描述

直接读取服务器 / 本地环境中的文件路径，触发浏览器下载。适用于下载应用内置文件、用户上传后存储的文件等场景。

#### 关键参数

| 参数名               | 类型  | 说明                                  |
|-------------------|-----|-------------------------------------|
| <code>path</code> | 字符串 | 本地文件的绝对路径或相对路径（相对当前 Python 脚本的运行目录） |

示例代码

```
from nicegui import ui

# 下载当前目录下的 main.py 文件（默认文件名: main.py）
ui.button('下载本地脚本', on_click=lambda: ui.download.file('main.py'))

# 下载本地 images 文件夹中的 pic.jpg，自定义下载文件名为 "我的图片.jpg"
ui.button('下载图片（自定义名称）', on_click=lambda: ui.download.file(
    path='images/pic.jpg',
    filename='我的图片.jpg',
    media_type='image/jpeg' # 明确指定图片类型
))

ui.run()
```

注意事项

- 路径需确保脚本有读取权限（避免权限不足导致下载失败）；
- 相对路径基于 Python 脚本的运行目录，而非前端页面路径，需注意目录结构一致性。

3. 从 URL 下载资源 (ui.download.from\_url())

功能描述

通过 URL 地址获取资源并触发下载，支持相对 URL（应用内部资源）和绝对 URL（外部资源），但有浏览器兼容性限制。

关键参数

| 参数名 | 类型  | 说明                         |
|-----|-----|----------------------------|
| url | 字符串 | 资源的 URL 地址（相对 URL 或绝对 URL） |

核心限制（重点关注）

- **相对 URL 优先支持：** 仅对应用内部的相对 URL（如 /logo.png、/static/files/data.zip）完全兼容，可正常下载并支持 filename 和 media\_type 参数；
- **绝对 URL 限制：**
  1. 浏览器会忽略 filename 和 media\_type 参数，仅遵循目标服务器返回的 Content-Disposition 头和文件类型；
  2. 部分文件类型（如图片、PDF、文本）可能被浏览器直接打开而非下载（需目标服务器配置 Content-Disposition: attachment 头）；
  3. 仅 .zip、.db 等非浏览器可预览格式，大概率触发下载；
  4. 跨域绝对 URL 可能因 CORS 政策限制导致下载失败（2.19.0 版本新增跨域下载警告）。
- **替代方案：** 若需下载绝对 URL 资源且确保触发下载，建议先用后端代码请求该 URL 获取资源，再通过 ui.download.content() 转发下载。

示例代码

```
from nicegui import ui

# 下载应用内部的相对 URL 资源（假设 /logo.png 是应用静态文件）
ui.button('下载应用 Logo', on_click=lambda: ui.download.from_url(
    url='/logo.png',
    filename='应用图标.png',
    media_type='image/png'
))

# 下载外部绝对 URL 资源（注意：filename 可能被浏览器忽略）
ui.button('下载外部 ZIP 文件', on_click=lambda: ui.download.from_url(
    url='https://example.com/files/data.zip' # 仅 .zip 等格式大概率触发下载
))

ui.run()
```

4. 下载原始数据 (ui.download.content())

功能描述

直接将字符串或字节数据作为文件内容下载，无需提前创建本地文件，适用于动态生成文件（如导出表格数据、生成日志、拼接文本内容等场景）。

关键参数

| 参数名     | 类型        | 说明                                   |
|---------|-----------|--------------------------------------|
| content | 字符串 / 字节串 | 要下载的原始数据（字符串会自动编码为 UTF-8 字节，字节串直接使用） |

示例代码

```
from nicegui import ui

# 1. 下载字符串数据（生成文本文件）
ui.button('下载文本文件', on_click=lambda: ui.download.content(
    content='Hello NiceGUI!\n这是动态生成的文本内容',
    filename='动态文本.txt',
    media_type='text/plain'
))

# 2. 下载字节数据（生成二进制文件，如 CSV）
import csv
from io import BytesIO

def generate_csv():
    # 动态生成 CSV 字节流
    output = BytesIO()
    writer = csv.writer(output)
    writer.writerow(['姓名', '年龄', '城市'])
```

```
writer.writerow(['张三', 25, '北京'])
writer.writerow(['李四', 30, '上海'])
output.seek(0) # 重置文件指针到开头
return output.read()

ui.button('下载 CSV 表格', on_click=lambda: ui.download.content(
    content=generate_csv(),
    filename='用户数据.csv',
    media_type='text/csv'
))

ui.run()
```

## 适用场景

- 导出数据库查询结果为 Excel/CSV;
- 生成动态报告（如日志摘要、统计数据）；
- 拼接用户输入内容为文件下载。

## 三、版本兼容性与核心注意事项

### 1. 版本要求

- 基础功能（所有方法）：NiceGUI ≥ 2.14.0;
- 跨域下载警告：NiceGUI ≥ 2.19.0（新增对跨域绝对 URL 下载的公告提示）。

### 2. 关键注意事项

1. **绝对 URL 下载限制**：如前文所述，`from_url()` 对绝对 URL 的支持有限，优先使用相对 URL 或后端转发方案；
2. **文件类型与浏览器行为**：即使指定 `media_type`，浏览器仍可能根据自身配置预览文件（如 PDF、图片），而非直接下载。若需强制下载，需后端配合设置 `Content-Disposition: attachment` 响应头；
3. **跨域资源共享 (CORS)**：下载外部绝对 URL 时，需目标服务器允许跨域请求（返回 `Access-Control-Allow-Origin` 头），否则会因浏览器安全策略拦截下载；
4. **大文件下载**：`ui.download` 直接通过前端触发下载，大文件（如 >100MB）可能导致浏览器卡顿，建议结合后端分块传输或断点续传方案。

## 四、核心总结

`ui.download` 是 NiceGUI 针对文件下载场景的“轻量化解决方案”，核心优势在于：

- API 简洁统一：三类下载场景共享参数，学习成本低；
- 无需前端编码：纯 Python 实现，无需编写 JavaScript 下载逻辑；
- 动态生成支持：`content()` 方法无需本地文件，直接处理内存数据，适配动态导出场景。

方法选择建议

| 下载场景             | 推荐方法                                | 备注                |
|------------------|-------------------------------------|-------------------|
| 本地文件 / 应用内置文件    | <code>ui.download.file()</code>     | 确保路径权限和目录正确性      |
| 应用内部静态资源（相对 URL） | <code>ui.download.from_url()</code> | 完全支持自定义文件名和文件类型   |
| 外部资源（绝对 URL）     | 后端转发 + <code>content()</code>       | 避免浏览器兼容性和 CORS 问题 |
| 动态生成数据（字符串 / 字节） | <code>ui.download.content()</code>  | 适配导出、报告生成等场景      |

NiceGUI 中 `ui.download` 的全维度解析

`ui.download` 是 NiceGUI 专为**文件下载功能**设计的核心 API，支持触发浏览器下载本地文件、动态生成的文件（如 CSV/Excel/JSON）、二进制数据等，无需手动配置 HTTP 响应头，是实现“导出数据、下载报表、保存文件”等功能的极简方式。

一、核心作用与适用场景

1. 核心作用

- 触发浏览器的文件下载行为，自动处理 MIME 类型、文件名、下载头；
- 支持本地文件、内存中的二进制数据、文本内容等多种数据源；
- 可与按钮、菜单等组件绑定，实现点击触发下载；
- 兼容所有现代浏览器，无需额外前端代码。

2. 典型适用场景

| 场景        | 示例                         |
|-----------|----------------------------|
| 下载本地静态文件  | 点击按钮下载应用内的 PDF 手册、图片等      |
| 导出动态生成的数据 | 将表格数据导出为 CSV/Excel/JSON 文件 |
| 下载二进制生成文件 | 动态生成的报表、压缩包、图片等            |
| 批量下载文件    | 打包多个文件为 ZIP 后触发下载          |

二、基本语法与使用方式

1. 基础语法

```
from nicegui import ui

# 语法1: 下载本地文件（指定文件路径）
ui.download(path: str, filename: str | None = None)
```

```
# 语法2: 下载内存中的二进制数据
ui.download(content: bytes, filename: str, mime_type: str | None = None)

# 语法3: 下载文本内容（自动编码为UTF-8）
ui.download(content: str, filename: str, mime_type: str | None = None)

# 完整参数（通用）
ui.download(
    target: str | bytes, # 本地文件路径 / 二进制数据 / 文本内容
    filename: str | None = None, # 下载时显示的文件名（可选，本地文件默认用原文件名）
    mime_type: str | None = None, # 文件MIME类型（可选，自动推断）
)
```

2. 核心参数详解

| 参数名       | 类型          | 取值说明                                                                                                       | 默认值   |
|-----------|-------------|------------------------------------------------------------------------------------------------------------|-------|
| target    | str / bytes | 核心目标：- 字符串：本地文件绝对 / 相对路径（如 ./data/report.pdf）- 字节串：内存中的二进制数据（如 b'csv content'）- 文本字符串：自动编码为 UTF-8 字节串      | 无（必填） |
| filename  | str / None  | 下载时浏览器显示的文件名（含扩展名）：- 本地文件：默认使用原文件名- 二进制 / 文本：必填（如 'data.csv'）                                              | None  |
| mime_type | str / None  | 文件的 MIME 类型（如 text/csv、application/json、application/pdf）：- 自动推断：根据文件名扩展名（如 .csv → text/csv）- 手动指定：覆盖自动推断结果 | None  |

3. 场景 1：下载本地静态文件

```
from nicegui import ui
import os

# 确保测试文件存在（示例：创建一个txt文件）
with open('test_file.txt', 'w', encoding='utf-8') as f:
    f.write('这是测试下载的本地文件内容')

# 点击按钮下载本地文件
ui.button(
    '下载本地文件',
    on_click=lambda: ui.download(
        path='test_file.txt', # 本地文件路径（相对/绝对）
        filename='自定义文件名.txt' # 可选：自定义下载文件名
    )
)

# 注意：本地文件路径需确保应用有权限读取
ui.run()
```

## 4. 场景 2：下载动态生成的文本文件（CSV/JSON）

```
from nicegui import ui
import json

# 示例1：导出CSV文件
def export_csv():
    # 动态生成CSV内容
    csv_content = '姓名,年龄,城市\n张三,25,北京\n李四,30,上海'
    # 下载文本内容（自动编码为UTF-8）
    ui.download(
        content=csv_content,
        filename='用户数据.csv',
        mime_type='text/csv; charset=utf-8' # 指定MIME类型（可选）
    )

# 示例2：导出JSON文件
def export_json():
    # 动态生成JSON数据
    json_data = {
        'users': [{'name': '张三', 'age': 25}, {'name': '李四', 'age': 30}]
    }
    # 转换为JSON字符串
    json_content = json.dumps(json_data, ensure_ascii=False, indent=2)
    # 下载JSON文件
    ui.download(content=json_content, filename='用户数据.json')

ui.button('导出CSV', on_click=export_csv)
ui.button('导出JSON', on_click=export_json)

ui.run()
```

## 5. 场景 3：下载二进制生成文件（图片 / Excel）

以生成并下载 Excel 文件（需安装 openpyxl）为例：

```
from nicegui import ui
from io import BytesIO
from openpyxl import workbook

# 动态生成Excel文件并下载
def export_excel():
    # 创建Excel工作簿
    wb = workbook()
    ws = wb.active
    ws.title = '用户数据'
    # 写入数据
    ws.append(['姓名', '年龄', '城市'])
    ws.append(['张三', 25, '北京'])
    ws.append(['李四', 30, '上海'])
    # 将Excel写入内存字节流
    buffer = BytesIO()
```

```
wb.save(buffer)
buffer.seek(0) # 重置指针到开头
# 下载二进制数据
ui.download(
    content=buffer.getvalue(), # 获取字节数据
    filename='用户数据.xlsx',
    mime_type='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
)

ui.button('导出Excel', on_click=export_excel)

ui.run()
```

### 三、关键特性与注意事项

#### 1. 路径处理规则

- **相对路径**: 相对于 NiceGUI 应用的启动目录（而非脚本所在目录），建议使用绝对路径避免歧义；
- **绝对路径**: 直接使用（如 `/home/user/data/file.pdf`），需确保应用进程有读取权限；
- **跨平台兼容**: Windows 路径需用 `\\` 或原始字符串（`r'c:\data\file.txt'`）。

#### 2. MIME 类型自动推断

NiceGUI 会根据文件名扩展名自动推断 MIME 类型，常见映射：

| 扩展名   | 自动推断的 MIME 类型                                                     |
|-------|-------------------------------------------------------------------|
| .txt  | text/plain                                                        |
| .csv  | text/csv                                                          |
| .json | application/json                                                  |
| .pdf  | application/pdf                                                   |
| .png  | image/png                                                         |
| .jpg  | image/jpeg                                                        |
| .xlsx | application/vnd.openxmlformats-officedocument.spreadsheetml.sheet |

若自动推断错误，可手动指定 `mime_type` 参数覆盖。

#### 3. 大文件下载优化

- 下载大文件（如 > 100MB）时，避免读取到内存（如 `content=open('big_file.zip', 'rb').read()`）；
- 推荐使用 `path` 参数直接指定文件路径，NiceGUI 会流式传输文件（无需加载到内存）：

```
# 大文件下载（流式传输，内存友好）
ui.button('下载大文件', on_click=lambda: ui.download(path='big_file.zip'))
```

## 4. 下载触发方式

除了按钮点击，还可通过其他事件触发下载：

```
from nicegui import ui

# 定时器触发下载（示例，实际按需使用）
def auto_download():
    ui.download(content='自动下载的内容', filename='auto_download.txt')

ui.timer(5, auto_download, once=True) # 5秒后自动触发下载
ui.label('5秒后将自动下载文件...')

# 菜单选项触发下载
with ui.menu_button('更多操作'):
    ui.menu_item('导出数据', on_click=lambda: ui.download(content='菜单触发下载',
        filename='menu_download.txt'))

ui.run()
```

## 5. 常见陷阱

- **文件权限问题**：应用进程无读取文件的权限，会导致下载失败（浏览器显示“下载失败”）；  
解决方案：检查文件权限，确保应用能访问目标文件。
- **内存溢出**：将大文件读取为字节串（`content=open('big.zip', 'rb').read()`）会占用大量内存；  
解决方案：使用 `path` 参数替代 `content` 参数。
- **文件名编码问题**：中文文件名乱码；  
解决方案：确保 `filename` 为 UTF-8 编码，或手动指定 `mime_type` 包含 `charset`：

```
ui.download(content='中文内容', filename='中文文件名.txt', mime_type='text/plain; charset=utf-8')
```

- **二进制数据指针问题**：使用 `BytesIO` 时未重置指针（`buffer.seek(0)`），导致下载的文件为空；  
解决方案：写入 `BytesIO` 后调用 `buffer.seek(0)`。

## 6. 与前端下载的区别

`ui.download` 是后端触发的下载（通过 HTTP 响应头 `Content-Disposition: attachment`），无需前端代码；

若需前端控制下载（如 Blob），可结合 `ui.run_javascript` 实现，但 `ui.download` 更简洁：

```
# 等效的前端下载（不推荐，仅作对比）
def js_download():
    js_code = """
    const content = '前端生成的内容';
    const blob = new Blob([content], {type: 'text/plain'});
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
```

```
a.href = url;
a.download = 'js_download.txt';
a.click();
URL.revokeObjectURL(url);
""""

ui.run_javascript(js_code)
```

```
ui.button('前端下载 (JS)', on_click=js_download)
```

## 四、实战场景示例（数据表格导出 CSV）

```
from nicegui import ui, app
import csv
from io import StringIO

# 模拟表格数据
table_data = [
    {'name': '张三', 'age': 25, 'city': '北京'},
    {'name': '李四', 'age': 30, 'city': '上海'},
    {'name': '王五', 'age': 28, 'city': '广州'},
]

# 显示数据表格
columns = [
    {'name': 'name', 'label': '姓名', 'field': 'name'},
    {'name': 'age', 'label': '年龄', 'field': 'age'},
    {'name': 'city', 'label': '城市', 'field': 'city'},
]
ui.table(columns=columns, rows=table_data).classes('w-full')

# 导出CSV（处理中文+表头）
def export_table_to_csv():
    # 创建StringIO写入CSV（支持中文）
    output = StringIO()
    writer = csv.writer(output, delimiter=',', quotechar='"', quoting=csv.QUOTE_MINIMAL)
    # 写入表头
    writer.writerow(['姓名', '年龄', '城市'])
    # 写入数据
    for row in table_data:
        writer.writerow([row['name'], row['age'], row['city']])
    # 获取CSV内容（重置指针）
    output.seek(0)
    csv_content = output.getvalue()
    # 触发下载
    ui.download(
        content=csv_content,
        filename='表格数据.csv',
        mime_type='text/csv; charset=utf-8'
    )

ui.button('导出CSV', on_click=export_table_to_csv).classes('mt-4')
```

```
ui.run()
```